

Recursive Directory Analytics Engine

Enterprise-scale file management systems require robust utilities to traverse complex, deeply nested directory structures for data auditing and storage optimization. Efficiently navigating these hierarchies demands high-performance recursive algorithms that can aggregate metadata while maintaining low memory overhead.

Implement a `DirectoryAnalyzer` class using the `pathlib` library and **recursion** to perform deep-tree analytics. The engine must traverse a root path, calculate the cumulative byte-size of all nested files, and provide precise frequency reporting based on file extensions. The system must be resilient to empty directories and complex sub-folder branching.

Example 1 Input:

```
MOCK_FS {"root": [true, ["a.txt", "docs"]], "root/a.txt": [false, 100], "root/docs": [true, ["b.txt", "c.py"]], "root/docs/b.txt": [false, 200], "root/docs/c.py": [false, 50]} SIZE
```

Output:

```
Total Size: 350
```

Explanation: The analyzer traverses the virtual file system starting from "root". It encounters a file `a.txt` (100 bytes) and a directory `docs`. Inside `docs`, it finds `b.txt` (200 bytes) and `c.py` (50 bytes). The recursive engine aggregates these values ($100 + 200 + 50$) to report a total storage footprint of 350 bytes.

Example 2 Input:

```
COUNT .txt
```

Output:

```
Count .txt: 2
```

Explanation: The recursive walker scans all levels of the hierarchy for files matching the `.txt` extension. It identifies `root/a.txt` and `root/docs/b.txt`, returning a cumulative count of 2.

Function Parameter:

- `root_path` (str): The entry point for the recursive traversal.
- `ext` (str): The file extension to filter for during the count operation.

Returns

- `dict`: A structured summary containing total size and root directory metadata.
- `int`: The frequency of files matching the specified extension.

Constraints

- The traversal **must** be implemented recursively.
- Utilize `pathlib.Path` for all file system interactions and path manipulations.
- Empty directories must be handled gracefully and contribute 0 bytes to the total.
- Maximum directory depth can reach 100 levels.

Input Format for Custom Testing Input from stdin will be processed as follows: The first line contains `MOCK_FS` followed by a JSON representation of the virtual file system. Subsequent lines contain commands: `SIZE`, `COUNT <ext>`, or `SUMMARY`.

Sample Case 0 Sample Input

```
MOCK_FS {"root": [true, ["temp"]], "root/temp": [true, ["data.log"]],  
"root/temp/data.log": [false, 1024]} SIZE
```

Sample Output

```
Total Size: 1024
```

Explanation The analyzer descends through "root" to "temp" and finally to "data.log". The file size (1024) is returned as the cumulative size of the branch.